

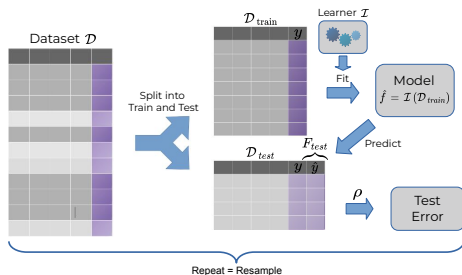
Outline

Recap: hold-out has problems

- ▶ Small training set $\rightarrow$ pessimistic bias
- ▶ Small test set $\rightarrow$ high variance in the estimate
- ▶ One unlucky split can mislead

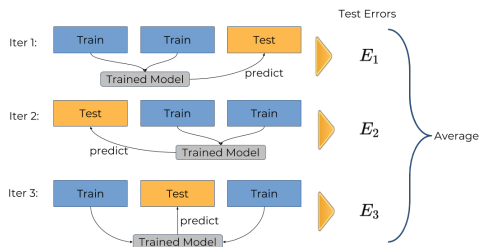
Resampling: repeatedly split into train/test, average the test errors.

- ▶ Use *most* of the data for training each time (low bias)
- ▶ Average across repetitions (low variance)
- ▶ Costs more compute, but pays for itself in reliability



k -fold CV

- ▶ Split $\mathcal{D}$ into k roughly equal folds
- ▶ Each fold serves as test set *exactly once*; the other $k - 1$ as training
- ▶ Get k test errors; report the mean (often the std too)
- ▶ Fraction $(k - 1)/k$ is used for training — e.g., 90% with 10-fold CV
- ▶ $k = 5$ or $k = 10$ are standard defaults
- ▶ $k = n$ is **leave-one-out (LOO) CV**



3-fold CV example.

Choosing k : bias vs variance

- ▶ **Larger** k : each fold uses more data for training $\Rightarrow$ less pessimistic bias
- ▶ **LOO** ($k = n$): nearly unbiased, but high variance — the n training sets overlap almost completely so estimates are correlated
- ▶ **Smaller** k : cheaper, more independent splits, but bigger bias

Practical defaults:

- ▶ Most cases: **5- or 10-fold CV**
- ▶ Small datasets: **repeated CV** (e.g., $10 \times$ 5-fold) — averages out fold-luck
- ▶ Avoid LOO unless you really need it (e.g., very small n with cheap learner)

Stratified CV

When classes (or values) are imbalanced:

- ▶ A random fold might miss a rare class entirely
- ▶ **Stratified CV**: keep the target distribution *within* each fold close to the overall distribution
- ▶ For classification: split each class separately, then join the per-class splits into folds
- ▶ Default in scikit-learn's `StratifiedKFold`

Bootstrap and subsampling

Subsampling (Monte Carlo CV):

- ▶ Repeated hold-out: draw a random train/test split many times, average
- ▶ Typical training fraction: 4/5 or 9/10
- ▶ More reps $\rightarrow$ less variance; smaller training fraction $\rightarrow$ more pessimistic bias

Bootstrap:

- ▶ Draw B training sets of size n *with replacement* from $\mathcal{D}$
- ▶ Test on **out-of-bag** (OOB) points $\mathcal{D}_{\text{test}}^b = \mathcal{D} \setminus \mathcal{D}_{\text{train}}^b$
- ▶ Each bootstrap training set contains $\approx 63.2\%$ of unique points
- ▶ Replicated points can cause artifacts; B632 / B632+ are variance-reducing extensions

Leave-one-object-out

When data is grouped (multiple obs per person, hospital, image, etc.):

- ▶ Standard i.i.d. assumption breaks
- ▶ Obs from the same object should be *either* all in train *or* all in test — never split
- ▶ Otherwise: the test set leaks information from training
- ▶ **Group k -fold** or **leave-one-object-out**: CV on objects, not individual rows

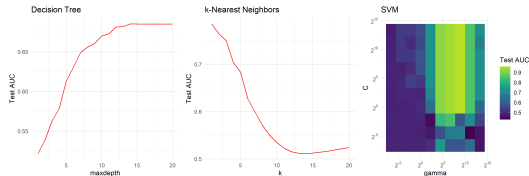
Parameters vs. hyperparameters

- ▶ **Model parameters θ** — optimized *during* training; an *output* of training
 - ▶ Linear regression: coefficients θ
 - ▶ Decision tree: splits and leaf values
- ▶ **Hyperparameters λ** — fixed *before* training; an *input* to training
 - ▶ Tree depth, k in k -NN, regularization λ , learning rate α
 - ▶ Often control model *complexity*

Finding good hyperparameters is called **tuning**.

Why tuning matters

- ▶ Many learners are very sensitive to HP values
- ▶ Same algorithm, different HPs $\Rightarrow$ vastly different generalization performance
- ▶ Defaults rarely optimal for your data

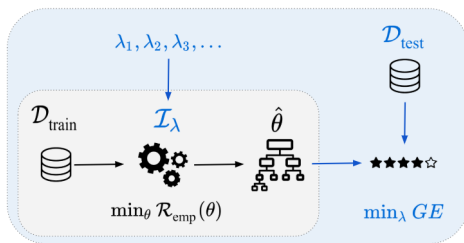


`mlbench.spiral` — different HPs yield big gaps.

The tuning problem (bilevel optimization)

Two nested optimizations:

1. **Inner loop:** given λ , train the model to find $\hat{\theta}$
2. **Outer loop:** search λ to minimize the estimated generalization error



HP types

Hyperparameters can be mixed:

- ▶ **Real-valued**: learning rate, regularization λ
- ▶ **Integer**: tree depth, k in k -NN, `mtry` in RF
- ▶ **Categorical**: split criterion (Gini / entropy), distance metric, optimizer choice

Often **hierarchically dependent**: e.g., *if* kernel is RBF, what is its bandwidth? — irrelevant when kernel is linear.

Grid search

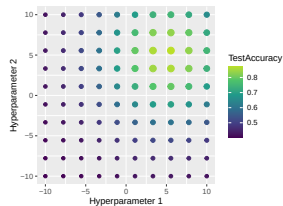
- ▶ Predefine candidate values for each HP
- ▶ Try *all combinations*
- ▶ Pick the best by CV score

Pros:

- ▶ Dead simple to implement
- ▶ All HP types supported
- ▶ Embarrassingly parallel

Cons:

- ▶ **Combinatorial explosion** in high-dim HP spaces
- ▶ Wastes evaluations on uninteresting regions
- ▶ Discretization is arbitrary



10×10 grid.

Random search

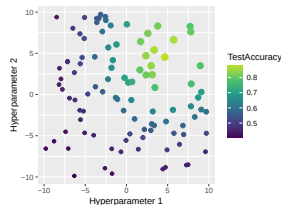
- ▶ Sample HP configurations uniformly from the region of interest
- ▶ Pick the best by CV score

Pros:

- ▶ Easy to implement, parallelizable
- ▶ **Anytime** — stop whenever budget runs out
- ▶ Each HP gets a unique value every trial — better coverage than grid in the few dimensions that matter

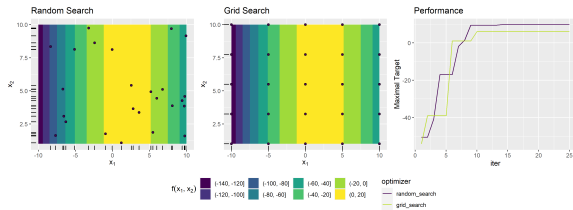
Cons:

- ▶ Still inefficient in high dim
- ▶ No use of information from past trials (cf. Bayesian opt)



100 random samples.

Why random often beats grid



- Suppose the target depends mostly on x_1 (irrelevant x_2)
- Grid: only 5 distinct x_1 values are tried (the rest are wasted on x_2)
- Random: each trial samples a fresh $x_1 \Rightarrow$ much better coverage of the relevant axis
- **Bergstra & Bengio (2012)** popularized this argument

Beyond grid and random

- ▶ **Bayesian optimization** (BO / SMBO): build a surrogate model of the HP→score function; pick the next config to maximize expected improvement. Much more sample-efficient.
- ▶ **Hyperband**: spend less compute on bad configs early; promising configs run longer. Good when training is expensive.
- ▶ **BOHB**: combines BO with Hyperband.
- ▶ Libraries: optuna, Ray Tune, Hyperopt, scikit-optimize.

Putting it all together

Typical workflow for an honest evaluation *with* tuning:

1. Hold out a final test set, set it aside — never touch it
2. On the remaining data, run CV-based tuning: search HP space, score by inner-CV mean
3. Refit the best HPs on the full inner data
4. Evaluate *once* on the held-out test set → unbiased estimate of GE

Why not just CV the chosen HPs?

Because tuning peeks at the CV score, which *also* acts as test data ⇒ optimistic bias.

The fix: **nested resampling** (L05b).

Wrap-up

1. Resampling beats single hold-out: lower variance, lower bias
2. k -fold CV ($k = 5$ or 10) is the default; stratify for imbalanced data
3. HPs are inputs to training; tuning is a bilevel optimization
4. Grid search: simple. Random search: usually better. BO / Hyperband: even better when expensive.
5. Tuning on CV makes the same CV score *biased* $\Rightarrow$ nested resampling (L05b).